

ADVANCED DIGITAL LOGIC CONTROL SYSTEM FOR MASTERMIND

Muhammad Mekaill Zaffar Murtaza, Muizz Aftab Warraich

Syed Babar Ali School of Sciences and Engineering, Lahore University of Management Sciences
Department of Electrical Engineering
LUMS, Sector U, DHA, Lahore Cantt. 54792, Lahore, Pakistan

ABSTRACT

Mastermind is a popular board game that involves players creating and decoding an array of four colors. In a regular digital logic implementation, this would involve using eight separate colors for eight LED lights. That is, four LED lights for the Game-Master and four for the decoding player. This would imply a total of 24 bits that may be toggled. This logic attempted to make this complex selection logic more simple and intuitive using sequential logic and more straightforward controls. Results showed that a full implementation of the game would benefit with such controls greatly, with only slight modification needed.

Index Terms— Mastermind, sequential logic, digital, analog, op-amp

1. INTRODUCTION

In a board game where the setup is as simple as placing a pin in a board, the justification for the ease of use of a digital logic implementation of the game becomes far more difficult.

This project attempts to simplify this normally complex implementation into a set of user friendly controls using a set of three different circuits: two selection circuits and a comparison circuit. Each selection circuit uses sequential logic to reduce the number of bits a user has to control from 12 to 4, making the circuit more user friendly and intuitive while preserving the core of the game.

The comparison logic then compares the outputs of the two selection circuits to show the decoding player how close they are to the real answer, and whether they have won. The focus of this project is on a simplified selection logic. Hence, a simple comparison has been used.

This is supplemented by a counter that counts the number of guesses made by the player, providing up to 9 guesses.

2. LITERATURE REVIEW

Mastermind is a code-breaking board game which one player generates a hidden combination of colored pegs while another player attempts to correctly guess the sequence. After each guess, feedback is provided indicating information

on the colors present in the pattern. Due to the gameplay mechanics relying on pattern generation, sequential guessing and comparison-based output feedback, it is a popular subject for implementation in digital logic orientated design projects.

2.1. Background

Originating in 1970, Mastermind is a well known classical board game. In its most recent iteration, published by Hasbro Inc., a plastic board allows a code-maker to set a hidden color code with 4 colored pins. The second player, the code-breaker, then has 12 attempts to decode this, with each attempt recorded on a 4×12 array of slots for colored pins. Two plastic sliders exist for each array, one red and one white. The white slider indicates that the code-breaker has selected a correct color of the available options, but this color is in the wrong position. The red slider indicates that a color is in the correct position. The sliders can be pulled all the way out to indicate that all four pins are in the correct position, meaning the code-breaker receives ten points [7].

2.2. Existing Literature

Existing research and implementations of Mastermind commonly rely on systems storing hidden code sequences, evaluating it against player-generated inputs. One notable implementation developed at Harvey Mudd College realized the game using FPGA architecture, where registers were used to maintain secret code sequences and player guesses while comparator logic determined matching conditions between stored and incoming patterns [3]. The project further incorporated LED-based output module to represent results.

Similarly, in a laboratory exercise at University of Southern California a Mastermind game was designed using digital logic. The implementation consisted of the usage of counters, multiplexers, registers, and decoding logic to coordinate gameplay. There was also an underlying emphasis on system states based on inputs and player color selections [2]. The USC implementation demonstrated how state-based implementations are prevalent in hardware-oriented puzzle and code-breaking games.

2.3. Relevant Technologies

While not commonly used to implement Mastermind, another area discussed in hardware game implementations is the existence of switch debouncing. Mechanical switches naturally generate oscillations, commonly referred to as switch bounce, during state transitions [1]. Posch explains that via RC filtering circuits alongside Schmitt-trigger hex inverters configurations it is possible to eliminate unstable pulses and ensure smooth clock pulses. Stable pulses are particularly important in because unintended pulses can corrupt stored game states and disrupt sequential progression.

Moreover, some use-cases also demonstrated the need for clock-gating methodologies as a means of selectively enabling specific sections of a digital system help tighten logic and prevent multiple state interference [5]. Although commonly associated with large-scale integrated systems, similar principles apply within smaller digital game architectures to control selective updates.

2.4. Summary

Overall, work concerning Mastermind implementations and other various game architectures demonstrates the suitability of digital electronics for realizing interactive puzzle-based games. Existing implementations consistently rely on state management, sequential addressing, and pattern comparison techniques. These studies collectively establish the theoretical and practical foundation for implementing board-game-inspired digital systems using combinational and sequential logic architectures.

3. BLOCK DIAGRAM

Please view Fig. 1.

4. DESIGN AND ANALYSIS

While Mastermind has been implemented in the past, little attention has been provided to create a more user friendly, hands-on control system.

When attempting to simplify a control system, it is essential to keep in mind that the new system must reduce the number of toggles the user must handle at a given time, while also not being more difficult to understand than the original. In the given system, namely Mastermind, the control system must toggle eight RGB LED lights, each of which control eight (2^3) different colors. These are further subdivided into two arrays of four lights for each player.

The components used were common anode RGB LED lights. Each light has three input bits, each controlling the colors red, blue and green. A 3 bit binary input could thus be used to select a range of color combinations. Thus, a row

of four RGB LED lights has a total of 32 different possible combinations.

Once the game-master uses their row of LEDs to set a hidden code, the player then has nine attempts to submit their guesses on what this hidden code may be. A separate array of LEDs indicates whether their guesses were right or wrong. In a standard implementation of Mastermind, these LEDs would not indicate the position of a correct guess; however, since this project focused explicitly on the control logic, a simple array of comparators was used to check the equality between the hidden code and the guess.

4.1. Rectification

On a high-level, before any logic was implemented, it became necessary to implement rectification to ensure the circuit functioned as expected, and prevent clock pulse signals from interfering with other parts of the circuit. This was done using four diodes connected between the +5V and ground connections in the selection logic circuits.

The interference occurs due to spikes in power, causing unexpected effects in different parts of the circuit [6].

An effect of this corrected later was that the selection circuit operated on 3.6V instead of 5V.

4.2. Selection Logic

While making a guess, users are simultaneously controlling two things: which LED in the array they want to configure and which color they would like to configure.

Thus, an effective simplification was to ensure that a user only toggled one LED at a time with a single switch. This reduced the number of bits from 12 per row to 4. That is, one for selecting the LED and three for each color.

For the LED toggle switch, two toggles were provided in total. One was provided for the game-master and one for the player. For the following, we will be following the connection path of the game-master toggle with the assumption that the player has an identical, yet separate sequential circuit for the same task.

This toggle was connected to a 4-bit binary decimal counter, which was configured to reset at 0100_2 , such that only the binary sequences from 0_{10} to 3_{10} were counted. The output of the counter were connected directly to an active-low 2-4 decoder, ensuring that only one output bit was 0V and all others were +5V. Each bit of this 4-bit output was sent into an OR-gate, which was directly connected to a clock pulse set at 1Hz. The output of the OR-Gate was linked to the CLK input of each flip-flop. Thus, only one flip-flop per array received a clock pulse at a time, while all others remained disabled. This was to overcome the unavailability of enable pins in all locally available flip-flop ICs.

For the three color selection bits, a set of three user friendly dials were implemented. These used op-amp com-

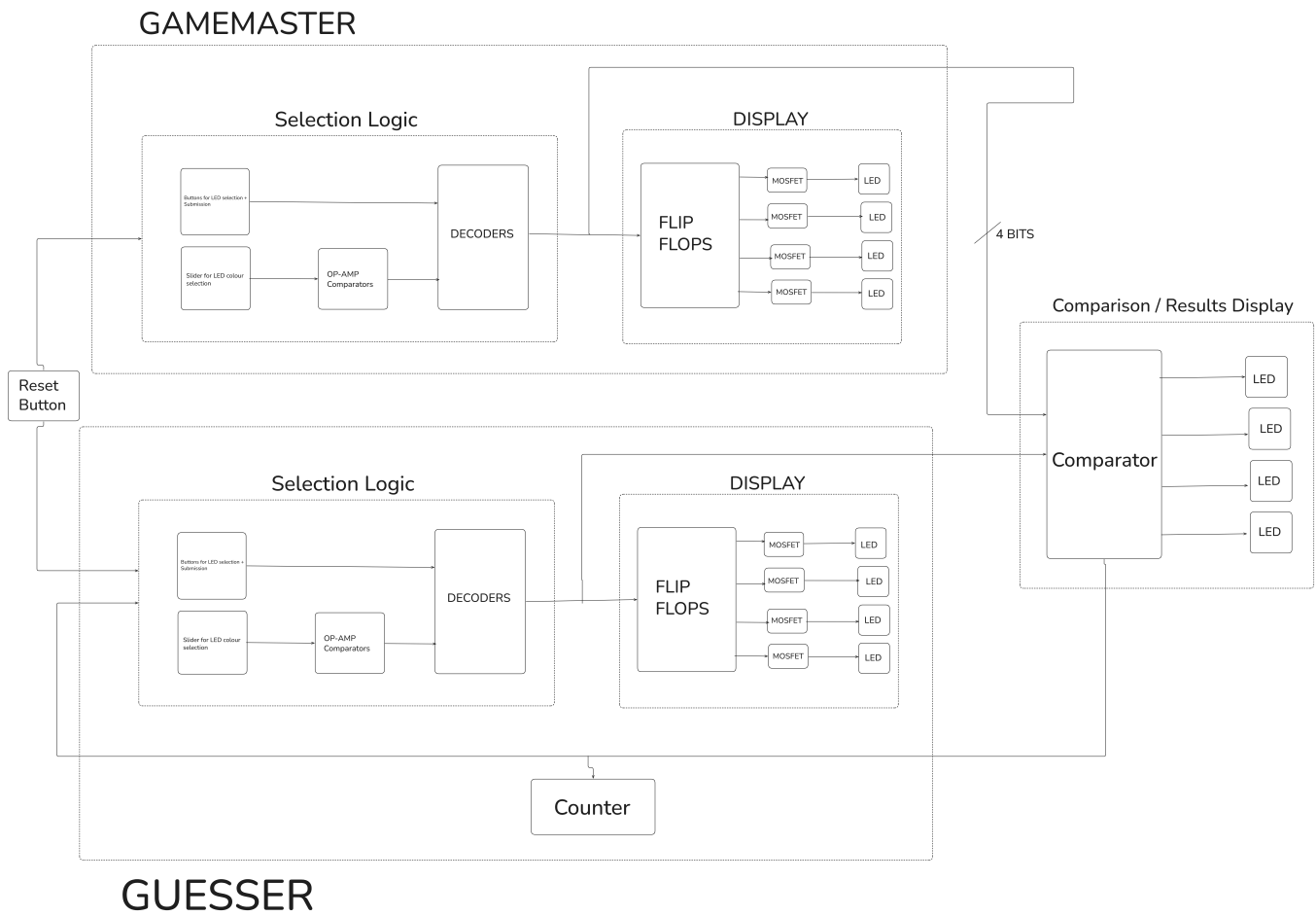


Fig. 1. Block diagram of circuit.

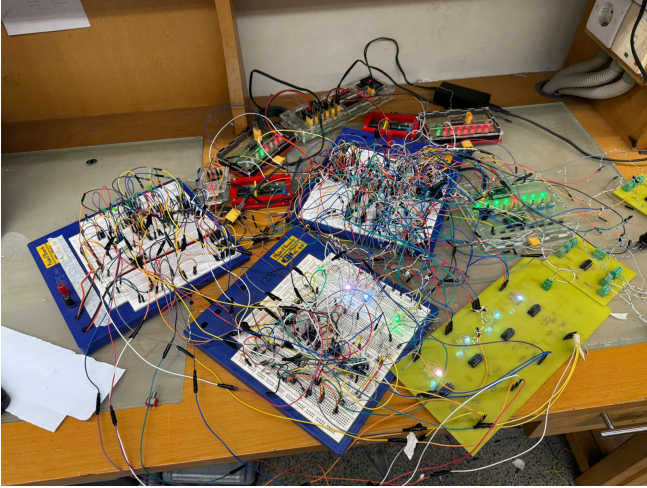


Fig. 2. Figure showing final hardware implementation.

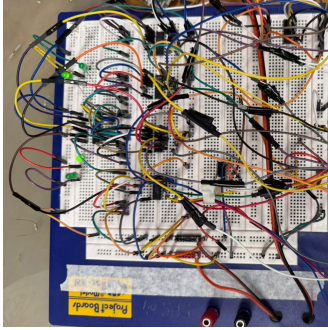


Fig. 3. Figure showing comparison logic circuit.

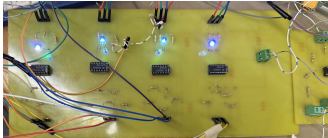


Fig. 4. Figure showing output and op-amp PCB.

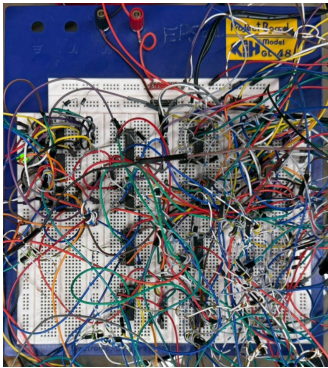


Fig. 5. Figure showing selection logic hardware.

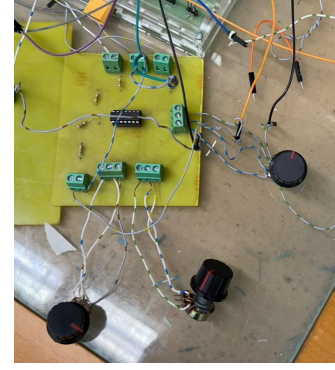


Fig. 6. Figure showing op-amp dials for color selection.

parator based analog to digital conversion to obtain the necessary three bit color sequence. This was done by setting the reference voltage of the op. The use of dials not only provided an aesthetic appeal, but was sufficiently memorable and different from the switches, allowing users to understand it faster and associate it directly with color control.

Thus, the finalised control system consisted of two sets of three dials, and two switches, reducing the overall number of control bits from 64 to 8. For a single player, the reduction was from 32 to 4.

Once the user has selected their 3-bit color sequence, the selected sequence is immediately sent into the input of the 3×4 array of flip-flops. The outputs of the flip-flops were connected to individual RGB LEDs, as well as a second op-amp comparator array. The RGB LEDs in this case are the two arrays of four LEDs, for the game-master and the player.

4.3. Op-Amp Comparator Setup

Op-amp comparators were crucial for two parts of the circuit: analog to digital conversion with dials and for correcting parasitic effects.

The analog to digital conversion dials provided an output that was either HIGH or LOW, depending on the input from variable resistors configured in the form of a set of dials. If the input was below a threshold of 2.5V, it was set to 0V. If it was above this level, it was set to 5V.

The following calculations were done for this,

$$V_{ref} = V_{cc} \times \frac{R_{gnd}}{R_{gnd} + R_{vcc}}$$

$$V_{ref} = 5V \times \frac{2.2k\Omega}{2.2k\Omega + 2.6k\Omega}$$

$$V_{ref} = 5V \times \frac{2.2}{4.4}$$

$$V_{ref} \approx 2.5V$$

2.5 V was selected for this conversion as it is the exact mid-point between +5V and 0V.

To correct parasitic effects and the voltage reduction due to the diodes, the output of each array of flip-flops was sent into an array of op-amp comparators. This array of comparators accounted for voltage fluctuations due to the parasitic effects from the rest of the circuit, which showed a significant reduction in voltages, below the threshold for HIGH and LOW signals that the ICs in the remaining circuit could use. This was calibrated via calculating the safe thresholds of voltages present at the anode based RGB pins which were around 4.3V when turned OFF, and around 2V when turned ON, thus 3.59 was taken as safe middle ground between these two values. The comparators set signals above 3.59 V to HIGH, and below 3.59V V to LOW, by setting their reference voltage to 3.59 V using the following calculations:

$$V_{ref} = V_{cc} \times \frac{R_{gnd}}{R_{gnd} + R_{vcc}}$$

$$V_{ref} = 5V \times \frac{5.6k\Omega}{2.2k\Omega + 5.6k\Omega}$$

$$V_{ref} = 5V \times \frac{5.6}{7.8}$$

$$V_{ref} \approx 3.589V$$

Thus, the reference resistances were set to 5.6 k Ω and 2.2 k Ω . This was to ensure that even with reduced voltage, the output signals remained either HIGH or LOW.

4.4. Comparison Logic

After the voltage correction from the flip-flop outputs, two 12-bit sequences were obtained, the hidden code and guess. These bits were sent directly into a single array of 4-bit digital comparators.

These are part of the Comparison circuit. The input of this circuit is a singular button, linked to an RC high-pass filter circuit to provide debouncing, ensuring that a smooth clock pulse was received by a final array of four flip-flops. The following calculations were done to set the parameters of the high-pass filter:

$$\tau = RC$$

$$\tau = 10 \text{ k}\Omega \times 100 \text{ nF}$$

$$\tau = 1 \text{ ms}$$

These flip-flops received a 4-bit sequence, with each bit coming from each of the four comparators. This sequence was connected to a final array of green LEDs to indicate to the player whether their guess was right or wrong. For instance, the bit sequence 1101 would light up three LEDs, indicating to the player that their guess is correct for the first, second and fourth LEDs, but incorrect for the third.

These flip-flops receive their clock pulse from a submit button. This button is connected to an RC high-pass filter, which is an effective way to design a debounce circuit [4].

The output of the filter is passed through a hex inverter to remove any remaining noise, producing a square pulse. This is used as a clock pulse for the flip-flop.

5. COMPONENTS USED

Please view Table 1. for hardware list.

The only software used extensively was Proteus 9 Professional v9.0, for both circuit simulation and PCB design.

6. PERFORMANCE ANALYSIS

To analyze the performance of the system, two main metrics are available: the quality of the output visible from the controls, and user satisfaction with the controls.

6.1. Output Quality Analysis

In the final system, performance in LED color output and comparison logic was reliable nearly perfect. Comparing the simulated results on Figs. 7, 8, 9, 14 and 15 to the hardware outputs on Figs. 10, 11, 12, 13 and 16 it can be seen that the logic is the same in both the simulation and on hardware. The only difference is the exact shade of color seen.

It should be noted that the colors (e.g. purple and white) were labeled arbitrarily and may not appear perfectly due to hardware constraints.

Figs 7 and 10 show the output when the game-master selects the combination Green-Turquoise-Turquoise-Green, 010 011 011 010. As can be seen, both the simulation and hardware show the same colors, apart from differences in shade. This is because Proteus uses a rendering engine to simulate nearly perfect colors, while real LEDs are impacted by room lighting and hardware limitations.

Figs 8 and 11, 10 and 15, 14 and 16 can similarly be compared to see colors in real life and in the simulation. Fig 8 and 11 show the bit combination 011 011 011 011 in Proteus and on hardware. All four colors are identical on Proteus, but on hardware, it can be seen that each LED appears to have a slightly different shade, despite showing an identical color.

Figs. 10, 11, 13, 16 and 17 all demonstrate the range of colors available as bit combinations of red, green and blue. As can be observed, each color is clearly distinguishable from other colors; however, some colors do not appear to be solid colors, unlike in the simulation in Proteus. For instance, the turquoise (blue-green) color visible in Figs. 10 and 11 shows LEDs with both blue and green, rather than a single, solid color. By contrast, Figs. 7 and 8 in Proteus show turquoise as a single, solid color.

Despite this, the combination of bits going into the comparator is exactly the same in both the simulation and in real life. That is, the core logic remains unchanged.

Table 1. List of Required Hardware

Category	Component/Specification	Quantity
Resistors	5.6 k Ω ($\frac{1}{4}$ -watt, $\pm 5\%$ Carbon)	20
	2.2 k Ω ($\frac{1}{4}$ -watt, $\pm 5\%$ Carbon)	20
	10 k Ω Potentiometer	6
Capacitors	1 nF (Ceramic, 50 V rated)	4
Semiconductors	Diode 1N4007	4
Integrated Circuits	Comparator IC LM339	10
	2-4 Decoder IC 74HC139	2
	Octal D-type Flip-Flop IC 74HC374	9
	OR Gate IC 74LS34	2
	4-Bit Comparator IC 74HC85	4
	4-Bit Binary Counter 74LS192	3
	Hex Schmitt Trigger Inverter 74HC14	1
Additional Components	RGB LED	8
	Green LED	4
	Push Button	1
	4-Bit BCD Display Module	1
	Clock pulse Generator Module	2

When considering the comparison circuit and comparing Figs 9 and 12, it can be seen that when the player and game-master select the same bit combination, all green LEDs light up, indicating that the player has won.

Comparing Figs 17 and 18 shows a case when the player and game master have only two color selections in common, hence only the third and fourth green LEDs light up.

Thus, this logic worked as expected with no deviations in output between the simulation and hardware.

6.2. User Satisfaction

Another important metric to consider is whether users truly found the controls intuitive. Five full-length tests were run with randomly selected students, with the student giving a verbal analysis of the ease of use of the system. Full length refers to a full game, from start-to end, with a hidden code set for them to decode.

In three tests, the users were able to effectively beat the game and found no issues with the control system.

In two tests, the users struggled to acclimate themselves with the controls, finding them tedious. The main complaint was that in each guess, every single LED had to be cycled over, which became tedious over time.

It can thus be suggested that the system is effective and intuitive, but requires changes to allow players to more easily cycle LEDs while preserving colors they do not wish to change.

6.3. Intermediary Results

During the setup of the circuit, tests were also run for the debounce logic. Fig. 19 shows an early result of the debounce

logic using the RC-filter and hex-inverter. The jagged edges on the top show noise. Fig. 20 shows the final, clean result that was used as a clock pulse for the comparison logic.

It should be noted the Proteus does not support clock pulses from buttons or RC-filters, so this system could not be simulated.

7. SOCIETAL RELEVANCE

Implementing board games such as Mastermind digitally with improved selection logic is extremely useful for society, as it provides a new way to experience traditional board games. It also improves improves existing designs that may already have selection logic that is far too complicated, making them more profitable and culturally relevant.

8. PROBLEMS FACED

Numerous problems were faced throughout the lab, those being parasitic effects, interference, hardware availability, hardware constraints and simulation constraints.

8.1. Interference

It was observed that as multiple sections of the circuit were connected to each other, as well as the clock pulse, they would seemingly interfere with the functionality of other parts of the circuit. For instance, the clock pulse of the game-master selection circuit began driving the player selection circuit, causing the counters and decoders to behave erratically. The only connection these two circuits had in common was the common ground and +5V connection.

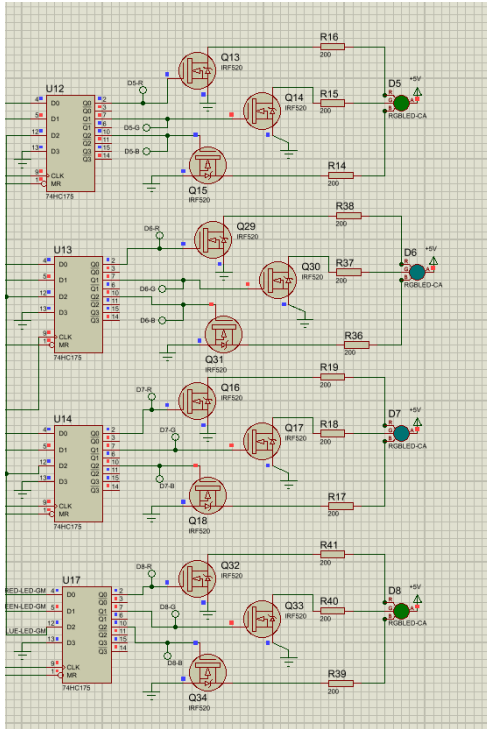


Fig. 7. Simulated Output of when the game-master selects the RGB color combinations Green-Turquoise-Turquoise-Green, 010 011 011 010.

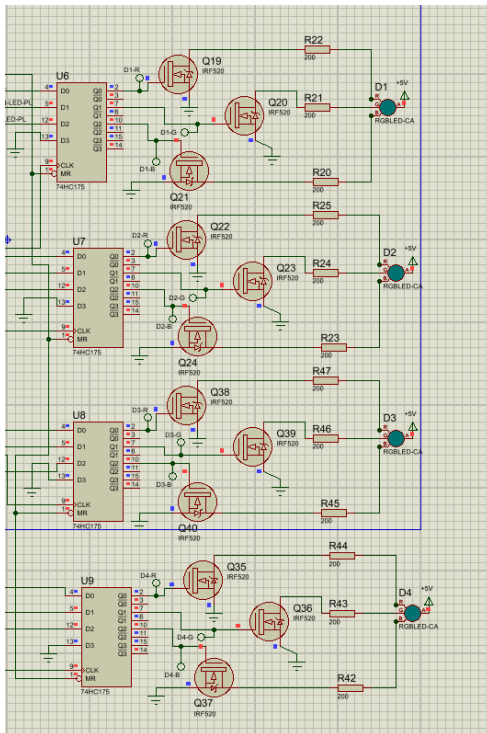


Fig. 8. Simulated Output of when the player selects the RGB color combinations Turquoise-Turquoise-Turquoise-Turquoise, 011 011 011 011.

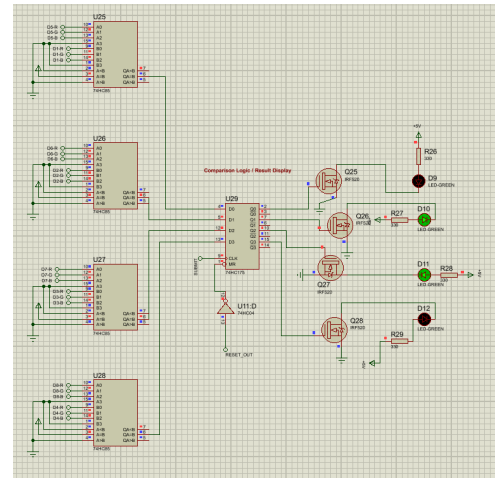


Fig. 9. Simulated comparison circuit output when the game-master selects 010 011 011 010 and the player selects 011 011 011 011. As expected, only the second and third LEDs are on.

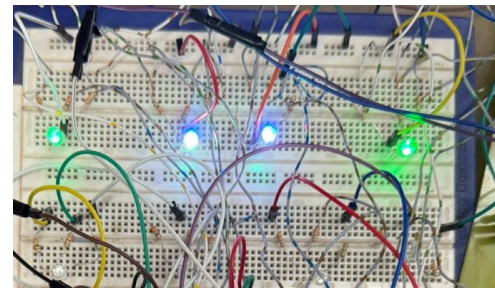


Fig. 10. Output of when the game-master selects the RGB color combinations Green-Turquoise-Turquoise-Green, 010 011 011 010.

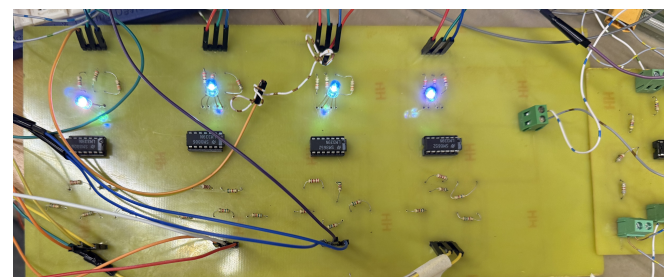


Fig. 11. Output of when the player selects the RGB color combinations Turquoise-Turquoise-Turquoise-Turquoise, 011 011 011 011.

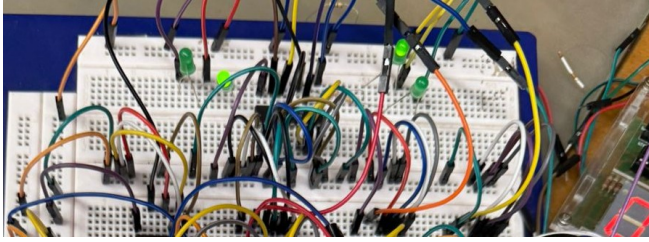


Fig. 12. Comparison circuit output when the game-master selects 010 011 011 010 and the player selects 011 011 011 011. As expected, only the second and third LEDs are on.

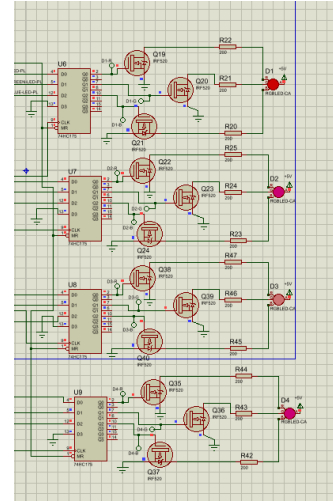


Fig. 15. Simulated output of player selecting red-purple-white-purple, 100, 101, 111, 101. The white appears to be slightly pink on Proteus.

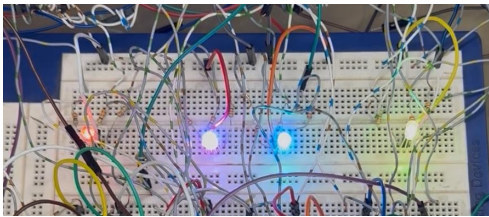


Fig. 13. Real output of game-master selecting red-purple-turquoise-yellow, 100 101 011 110

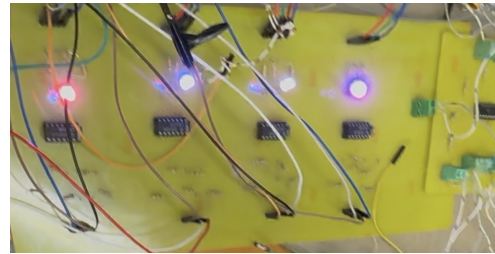


Fig. 16. Real output of player selecting red-purple-white-purple, 100, 101, 111, 101.

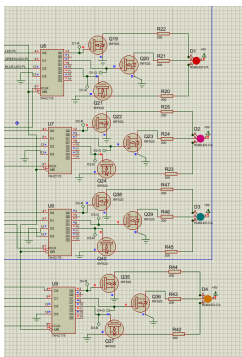


Fig. 14. Simulated output of game-master selecting red-purple-turquoise-yellow, 100 101 011 110

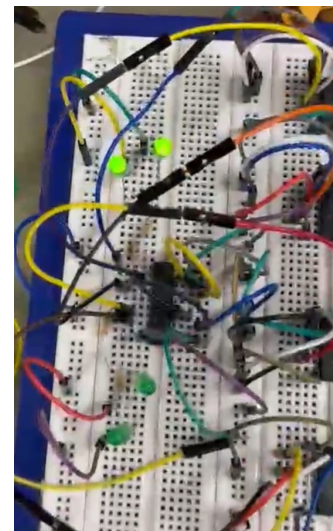


Fig. 17. Output when the third and fourth LEDs of the player and game master match.

configured to work only with the portable power supply. Attempts to use both supplies resulting in unexpected voltage gains across the circuit. Another attempted correction was the use of hex-inverters. However, it was found that some of the outputs had voltages far below the threshold accepted by a hex inverter.

The final proposed solution was the use of op-amp comparators with a reference voltage between the two LOW and HIGH signals, to correctly set the voltages to either 5V or 0V, in line the remaining circuit.

8.3. Hardware Availability

A major problem faced was that multiple required components were neither available in the department, nor in any local store.

The foremost of these was the Quad D-type Flip Flop IC (74HC175). The only available counterpart was the octal D-type flip-flop, which did not contain an enable pin. This increased the workload as more soldering was required for the larger IC. Moreover, it meant that the circuit had to be configured to override the clock pulse to provide an alternative to the enable pin, functioning as a clock-gate [5].

This flip-flop IC also did not have a reset pin, similar to the quad-type. This meant that the reset logic in the simulation could no longer be created, as there was no direct way to reset the flip-flops.

8.4. Hardware Constraints

Numerous hardware constraints were observed. The foremost of these was the unreliability of the RGB LED lights. Their colors were imperfect, but usable. This can be seen in the results, as LED color varies and is often not a single, solid color.

Secondly, the stock of jumper cables and wires was often insufficient, and the wires rarely had enough length to join parts of the circuit.

8.5. Simulation Constraints

The Proteus simulation produced some imperfect results that did work in real life. That is, it did not allow an RC filter to be used as a debouncer to power the clock pulse of any IC. Thus, the RC filter and button could not be included in the simulation.

9. SUGGESTION FOR IMPROVEMENT

9.1. Project Improvements

To further improve the circuit, it is recommended that the loading option of the counter be used to allow players to jump

between LEDs. This will allow greater flexibility while allowing players to avoid changing the colors of LEDs between guesses, if they have guessed correctly.

Moreover, for a complete implementation of Mastermind, the current comparison logic could be replaced by a parallel counting comparison logic that counts how many selections are correct and indicates this to the player without disclosing the exact position. A second logic should also check for colors that match but are not correctly positioned. This would require two separate rows of LEDs. If directly implemented, it will increase complexity significantly.

Thirdly, a 10×4 array of RGB LEDs could be added to indicate to players what past guesses they made. Thus, they will no longer have to remember their exact guess and can look back. This is again part of the full implementation, but will increase complexity significantly, as the sequential logic required for each row is expected to take at least four flip-flops each.

9.2. Laboratory Improvements

It is recommended that a greater number of ICs are made available that are not locked away, as the laboratory stuff was often busy or unavailable, restricting access to supplies. Moreover, longer, higher quality wires and jumper cables should be provided.

10. REFERENCES

- [1] “Debounce a Switch Time Constants for Common Resistor and Capacitor Values: Capacitor (μ F) Time Constant (ms).”
- [2] *EE 209 Lab 3 - Mind over Matter*.
- [3] C. Larsen and W. Berriel, *Mastermind: The Game Final Project Report*, 2002.
- [4] M. Posch, “Designing an RC Debounce Circuit,” *Maya’s Programming & Electronics Blog*, Jun. 26, 2018. [Online]. Available: <https://mayaposch.wordpress.com/2018/06/26/designing-an-rc-debounce-circuit/>
- [5] “The Ultimate Guide to Clock Gating,” *AnySilicon*, Feb. 3, 2021. [Online]. Available: <https://any silicon.com/the-ultimate-guide-to-clock-gating/>
- [6] Y. Luo, J. Wang, H. Yuan, T. Wei, Y. Huang, F. Wu, Y. Liu, and X. Chu, “Calculation of the Worst-Case Voltage Noise for a Power Distribution Network Based on Ramp Current,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 6, pp. 2797–2809, 2024, doi: 10.1109/TCSI.2024.3362253.
- [7] Hasbro SA, *Mastermind: The Classic Code Cracking Game!*. Hasbro SA, Mar. 2011.